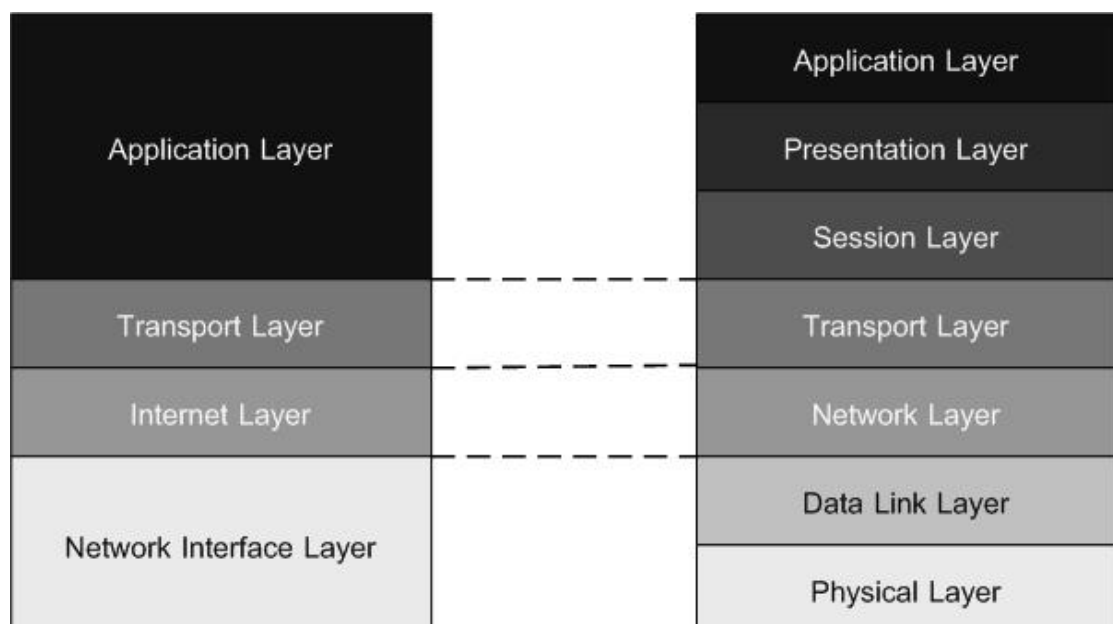


## บทที่ 2

# โปรโตคอลทีซีพี/ไอพี

### 2.1 ความเป็นมาของโปรโตคอล ทีซีพี/ไอพี

ทีซีพี/ไอพี เป็นมาตรฐานการรับส่งข้อมูลระหว่างเครื่องคอมพิวเตอร์สองระบบ ที่มีขึ้นเมื่อกระทรวงกลาโหมสหรัฐฯ หรือ Department Of Defense (DOD) ทำการทดลองในปี ค.ศ.1969 เชื่อมโยงคอมพิวเตอร์ทางทหารของแต่ละหน่วย ซึ่งเป็นคอมพิวเตอร์ต่างชนิดกันให้สามารถติดต่อรับส่งข้อมูลกันได้ โครงการนี้มีชื่อว่า Advanced Research Projects Agency Network หรือ ARPANET ซอฟต์แวร์ที่ใช้ควบคุมการรับส่งข้อมูล ARPANET ประกอบด้วยส่วนหลัก ๆ 2 ส่วนคือ ทีซีพี (Transmission Control Protocol หรือ TCP) และ ไอพี (Internet Protocol หรือ IP) ซึ่งทีซีพีมีหน้าที่ตรวจสอบการรับส่งข้อมูลระหว่างคอมพิวเตอร์ระหว่างผู้รับ และผู้ส่ง ให้ได้รับข้อมูลถูกต้องครบถ้วน ส่วนไอพีจะมีหน้าที่เลือกเส้นทางที่ใช้รับส่งข้อมูล ผ่านระบบเครือข่าย และตรวจสอบที่แอดเดรสของผู้รับ เรียกว่าไอพีแอดเดรส (IP Address) ต่อมาในปี ค.ศ. 1983 ทีซีพี/ไอพี ถูกกำหนดให้เป็นมาตรฐานการรับส่งข้อมูลของกระทรวงกลาโหมสหรัฐฯ และได้รวมเป็นส่วนหนึ่งของระบบปฏิบัติการยูนิกซ์ ส่งผลให้มีการใช้งานกันอย่างแพร่หลาย ในปัจจุบันมีการใช้งานกันอยู่แทบทุกเครือข่าย ไม่ว่าจะเป็นเครือข่ายเฉพาะที่ หรือเครือข่ายในบริเวณกว้าง ทีซีพี/ไอพีเชื่อมกลุ่ม เครือข่ายย่อยเข้าด้วยกันเป็นเครือข่ายขนาดใหญ่ หรืออินเทอร์เน็ต



รูปที่ 2-1 แสดงการเปรียบเทียบเลเยอร์ของโอเอสไอกับเลเยอร์ของ ทีซีพี/ไอพี

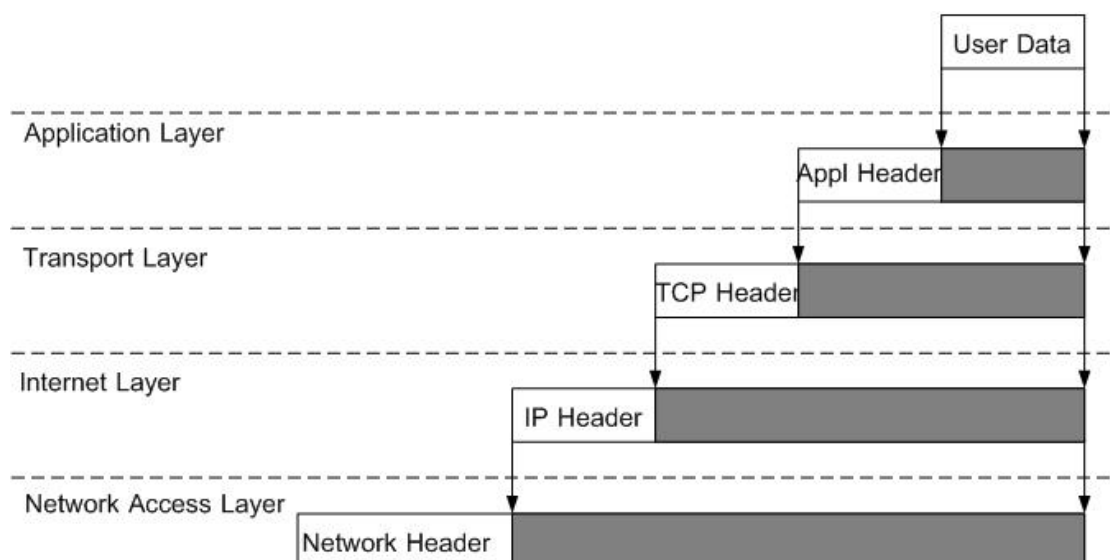
## 2.2 การเชื่อมต่อของโปรโตคอล ทีซีพี/ไอพี (TCP/IP Linking)

ทีซีพี/ไอพี (Transmission Control Protocol/Internet Protocol) เป็นโปรโตคอลในการสื่อสารในระบบอินเทอร์เน็ต และ อินทราเน็ต การทำงานของทีซีพี/ไอพี สามารถเปรียบเทียบกับโมเดลอ้างอิงโอเอสไอ (Open System Interconnection Reference Model: OSI) ตามมาตรฐานไอเอสโอ (International Organization for Standardization: ISO) ได้ดังรูปที่ 2-1

แต่ในระดับชั้นของ ทีซีพี/ไอพี (TCP/IP) จะมีการทำงานที่แตกต่างกัน ตั้งแต่การติดต่อกับแอปพลิเคชัน จนกระทั่งแปลงเป็นสัญญาณส่งไปตามสายสัญญาณ ซึ่งการทำงานในแต่ละระดับชั้นของ ทีซีพี/ไอพี มีดังตารางที่ 2-1

| ชื่อระดับชั้น                                             | หน้าที่                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. ชั้นแอปพลิเคชัน<br>(Application Layer)                 | ชั้นนี้รองรับการทำงานของแอปพลิเคชันต่าง ๆ ที่ทำงานเป็นโปรเซสอยู่ในเครื่องต้นทาง และ ปลายทาง โดยจัดการเชื่อมต่อระหว่างโปรเซส หรือแอปพลิเคชันที่อยู่ต่างเครื่องกัน โดยการทำงานของแอปพลิเคชันต่าง ๆ มีการติดต่อกันตามแต่ละโปรโตคอล เฉพาะแล้วแต่แอปพลิเคชัน ที่ใช้งานซึ่งจะขอบริการจากชั้นทรานสปอร์ตอีกทีหนึ่ง                                                                         |
| 2. ชั้นทรานสปอร์ต<br>(Transport Layer)                    | มีการสร้างการเชื่อมต่อกันระหว่างแอปพลิเคชันแบบ end-to-end โดยจุดที่เชื่อมต่อกันเพื่อรับส่งข้อมูลนี้เรียกว่า พอร์ต(Port) หรือ ซ็อกเก็ต (Socket) ในชั้นนี้มีการบริการหลักอยู่ 2 แบบคือ Connection Oriented โดยเรียกผ่านโปรโตคอล ทีซีพี (TCP: Transmission Control Protocol) และ Connectionless ซึ่งเรียกผ่านโปรโตคอลยูดีพี (UDP: User Datagram Protocol) ซึ่งจะกล่าวถึงในหัวข้อถัดไป |
| 3. ชั้นอินเทอร์เน็ต<br>(Internet Layer)                   | ชั้นนี้มีหน้าที่ส่งผ่านข้อมูลระหว่างเครือข่าย โดยมีโปรโตคอลที่ทำงานเป็นกลไกสำคัญในการส่งผ่านข้อมูลไปยังเครือข่ายใด ๆ ในอินเทอร์เน็ต คือ ไอพี(IP) ซึ่งกล่าวถึงในหัวข้อถัดไป นอกจากนี้ในชั้นนี้ยังมีโปรโตคอลทำงานอยู่ด้วยอีก 2 ชนิดคือ ไอซีเอ็มพี (ICMP) และ เออาร์พี (ARP)                                                                                                          |
| 4. ชั้นเน็ตเวิร์กอินเตอร์เฟซ<br>(Network Interface Layer) | ทำหน้าที่ในการแปลงข้อมูลให้อยู่ในรูปที่เหมาะสมกับเครือข่ายแต่ละแบบ ซึ่งแตกต่างกันออกไป และแปลงเป็นสัญญาณ ไฟฟ้าส่งไปยังเครือข่าย                                                                                                                                                                                                                                                    |

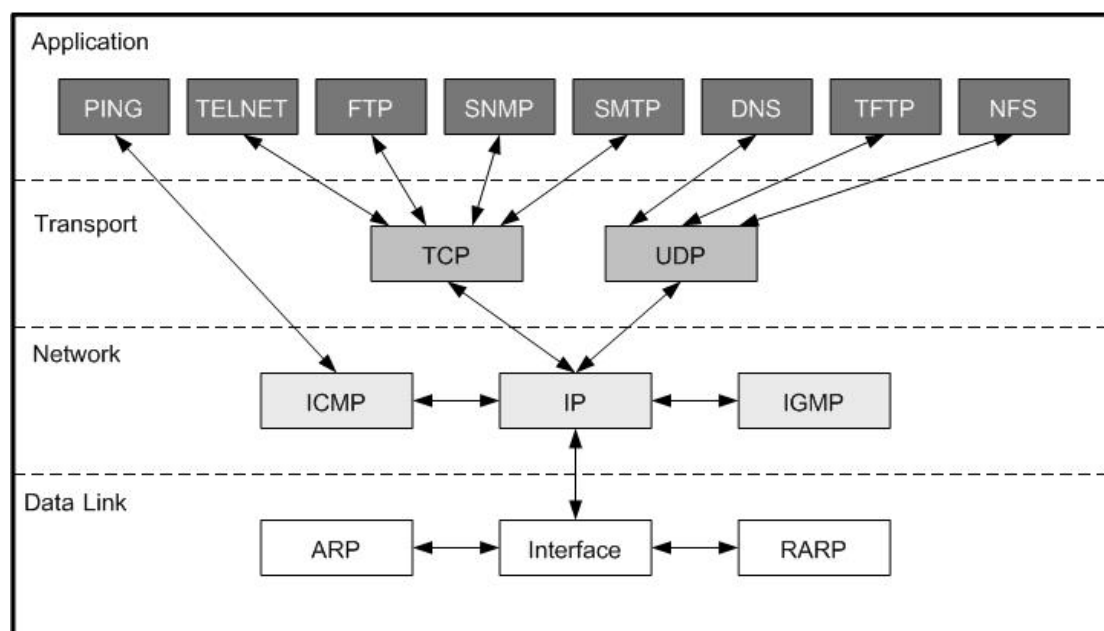
ตารางที่ 2-1 การทำงานของแต่ละระดับชั้นของ ทีซีพี/ไอพี



รูปที่ 2-2 แสดงการห่อหุ้มข้อมูลตามลำดับโปรโตคอลแตก

### 2.3 โปรโตคอลแตก

การทำงานตามโปรแกรมประยุกต์หนึ่ง ๆ ไม่ได้ใช้โปรโตคอลพร้อมกันทั้งหมด หากแต่ใช้เพียงโปรโตคอลที่สัมพันธ์กันไปในแต่ละระดับชั้นของแบบอ้างอิงตัวอย่างเช่นการใช้งาน เทลเน็ต (Telnet) ที่ จะอาศัย ทีซีพี/ไอพี (TCP/IP) ตามลำดับ การซ้อนทับของโปรโตคอลจากระดับชั้นบนไปชั้นล่าง เรียกว่า โปรโตคอลแตก (Protocol Stack) แสดงดังรูปที่ 2-3



รูปที่ 2-3 โปรโตคอลแตกของ ทีซีพี/ไอพี

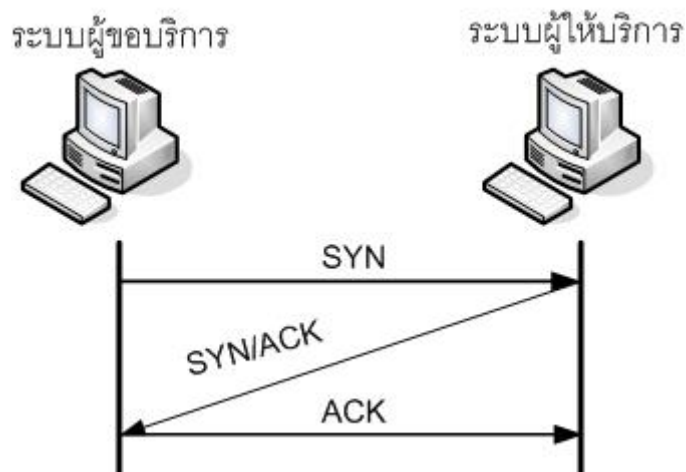
ไอพีซึ่งอยู่ในระดับชั้นเน็ตเวิร์กตามรูปที่ 2-3 เป็นแกนสำคัญของโปรโตคอลแสดงเนื่องจากทั้งทีซีพี(TCP) และ ยูดีพี (UDP) ต้องใช้ไอพีเลือกเส้นทางส่งแพ็กเก็ต ในระดับชั้นเน็ตเวิร์ก ยังมีไอซีเอ็มพี (ICMP) สนับสนุนการทำงานของไอพีเพื่อรายงานข้อผิดพลาดที่เกิดขึ้น เนื่องจากการส่งแพ็กเก็ต และมีไอจีเอ็มพี (IGMP) ดูแลการจัดกลุ่มโฮสต์ในเครือข่ายมัลติคาสต์

ระดับชั้นทรานสปอร์ตมีสองโปรโตคอลสำคัญคือ ทีซีพีและยูดีพี แอปพลิเคชันจะเลือกใช้ทีซีพีหรือยูดีพีตามลักษณะงาน

โปรโตคอลระดับล่างถัดจากไอพีได้แก่ โปรโตคอลระดับเดทาลิงก์ซึ่งกำหนดการทำงานตามเทคโนโลยีเครือข่ายที่ใช้ งาน เช่น โปรโตคอลระดับเดทาลิงก์ ซึ่งกำหนดการทำงานตามมาตรฐานอีเทอร์เน็ต ในระดับชั้นนี้มีโปรโตคอลในชุดของทีซีพี/ไอพี ทำหน้าที่สนับสนุนการทำงานอยู่สองโปรโตคอล คือ เออาร์พีและอาร์เออาร์พี ทั้งสองโปรโตคอลจะทำหน้าที่แปลงค่าระหว่างไอพีแอดเดรสกับฮาร์ดแวร์แอดเดรส

## 2.4 โปรโตคอลทีซีพี (TCP: Transmission Control Protocol)

การทำงานที่สำคัญอย่างหนึ่งที่สำคัญของโปรโตคอลทีซีพีคือการทำ“3-way Handshake” ซึ่งเป็นกระบวนการเริ่มต้นในการสร้างการเชื่อมต่อในชั้นทรานสปอร์ต กล่าวคือ ในการติดต่อกันระหว่างระบบในเครือข่ายต้องมีการสร้างการเชื่อมต่อ ไปยังระบบที่ ให้บริการก่อนโดยผู้ขอบริการส่งสัญญาณ SYN เพื่อขอบริการ จากนั้นผู้ให้บริการจะส่งสัญญาณ ACK เพื่อตอบรับการเชื่อมต่อที่ร้องขอมาจึงสามารถรับส่งข้อมูลได้ แสดงดังรูปที่ 2-4



รูปที่ 2-4 แสดงการทำ 3-way Handshake

การเชื่อมต่อแบบ 3-way Handshake นี้เป็นการตรวจสอบความพร้อมของทั้งฝ่ายส่ง และฝ่ายรับ และเป็นการกำหนดค่าเริ่มต้นของพารามิเตอร์ต่าง ๆ ของทั้งสองฝ่ายให้ตรงกัน หลังจากกระบวนการทำ 3-way Handshake สิ้นสุด ทั้งสองฝ่ายจึงสามารถรับและ ส่งข้อมูลซึ่งกันและกันได้

ดังนั้นโปรโตคอลที่ซีพีจึงเป็นโปรโตคอลที่มีการรับส่งข้อมูลแบบ “Connection Oriented” ทำให้การทำงานของซีพีมีความน่าเชื่อถือมากขึ้น หน้าที่การทำงานของซีพี ในการรับส่งข้อมูลมีหน้าที่หลัก 6 ข้อคือ

1. ควบคุมการรับส่งข้อมูล (Basic Data Transfer)
2. ความน่าเชื่อถือในการรับส่งข้อมูล (Reliability)
3. ควบคุมการไหลของข้อมูล (Flow Control)
4. การทำมัลติเพล็กซ์ (Multiplex)
5. ควบคุมการเชื่อมต่อ (Connection control)
6. ความปลอดภัยในการรับส่งข้อมูล (Security)

ส่วนประกอบของซีพีเฮดเดอร์

|                        |          |       |                  |    |  |
|------------------------|----------|-------|------------------|----|--|
| 0                      |          | 15 16 |                  | 31 |  |
| source port            |          |       | destination port |    |  |
| sequence number        |          |       |                  |    |  |
| acknowledgement number |          |       |                  |    |  |
| offset                 | reserved | code  | window size      |    |  |
| checksum               |          |       | urgent pointer   |    |  |
| options + pad          |          |       |                  |    |  |
| data                   |          |       |                  |    |  |

รูปที่ 2-5 ซีพีเฮดเดอร์

- source port ขนาด 16 บิต: เป็นหมายเลขพอร์ตของบริการที่เครื่องต้นทาง
- destination port ขนาด 16 บิต: เป็นหมายเลขพอร์ตของบริการเครื่องปลายทาง
- sequence number ขนาด 32 บิต: เป็นหมายเลขที่บอกลำดับของการรับส่งข้อมูลของเครื่องที่ต้องการขอส่งข้อมูล
- acknowledgement Number ขนาด 32 บิต: เป็นหมายเลขที่บอกลำดับของการรับส่งข้อมูลที่ฝั่งรับข้อมูลปกติค่าของ Acknowledgement Number จะมีค่าเท่ากับ Sequence Number (ของอีกฝั่ง) + 1 เสมอ
- data offset ขนาด 4 บิต: เป็นตัวบอกค่าออฟเซตของข้อมูล เพราะซีพีนั่นไม่มีการกำหนดความยาวที่แน่นอนของข้อมูล จึงต้องมีออฟเซตเป็นตัวบอก
- code ประกอบด้วย 6 บิตย่อยแต่ละบิตย่อยมีขนาด 1 บิต ได้แก่

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| U | A | P | R | S | F |
| R | C | S | S | Y | I |
| G | K | H | T | N | N |

- URGent ถ้าบิตนี้เป็น “1” หมายความว่า Urgent pointer บรรจุตำแหน่งข้อมูลที่ต้องรีบดำเนินการเร่งด่วนก่อน
  - ACKnowledgement ถ้าบิตนี้เป็น “1” หมายถึงเป็นเชกเมนต์ตอบรับ โดยตอบอ้างอิงเลขลำดับตามที่กำหนดในฟิลด์ Acknowledgement number
  - PuSH ถ้าบิตนี้เป็น “1” หมายความว่าทันทีที่สถานีปลายทางได้รับเชกเมนต์ ต้องรีบส่งข้อมูลไปยังโปรโตคอลประยุกต์ทันทีโดยไม่ต้องรอให้บัฟเฟอร์เต็ม
  - ReSeT ถ้าบิตนี้เป็น “1” หมายถึงให้ยกเลิกการเชื่อมต่อ เนื่องจากอาจมีความผิดปกติเกิดขึ้นระหว่างคู่สถานีที่กำลังติดต่อกันอยู่ หากจำเป็นต้องส่งข้อมูลระหว่างกันอีกก็ต้องเริ่มสถาปนาการเชื่อมต่อใหม่
  - SYNchronize ถ้าบิตนี้เป็น “1” หมายถึงขอเริ่มต้นสถาปนาการเชื่อมต่อและเมื่อการสถาปนาเสร็จสิ้น บิตนี้จะถูกกำหนดให้เป็น “0” หลังจากนั้นจึงสามารถส่งผ่านข้อมูลระหว่างกันได้
  - FINish ถ้าบิตนี้เป็น “1” หมายถึงขอจบการเชื่อมต่อ
- windows size ขนาด 16 บิต: สถานีปลายทางใช้ฟิลด์นี้แจ้งขนาดบัฟเฟอร์ที่มีอยู่ (หน่วยเป็นไบต์) สถานีที่ติดต่อด้วยต้องไม่ส่งข้อมูลเกินค่านี้
  - checksum ขนาด 16 บิต: ผลรวมตรวจสอบความถูกต้องของเชกเมนต์โดยคำนวณทั้งเฮดเดอร์และข้อมูล
  - urgent pointer ขนาด 16 บิต: พอยเตอร์ชี้ตำแหน่งไบต์ข้อมูลที่ต้องดำเนินการเร่งด่วนที่ต้องการให้โปรแกรมประยุกต์ดำเนินการทันที ค่าที่บรรจุในฟิลด์นี้จะมีความหมายก็ต่อเมื่อแฟล็ก URG ถูกเซตเป็น “1”
  - options ขนาดแปรเปลี่ยนได้: ใช้กำหนดงานเพิ่มเติมให้กับทีซีพีซึ่งจะมีหรือไม่ก็ได้ หากฟิลด์ offset หากมีค่าเป็น 5 แสดงว่ามีเฮดเดอร์ขนาด 20 ไบต์ซึ่งหมายถึงไม่ใช่ออฟชั่น
  - pad ขนาด 0 ถึง 24: ใช้เป็นส่วนที่ทำให้ขนาดของออฟชั่นเป็นจำนวนเท่าของ 32 บิต

## 2.5 โพรโทคอลยูดีพี (UDP: User Datagram Protocol)

โพรโทคอลยูดีพีเป็นโพรโทคอลในการติดต่อสื่อสารในชั้นทรานสปอร์ต (Transport Layer) การทำงานคล้ายกับทีซีพีมาก คือ จัดการเกี่ยวกับการสื่อสารระหว่างเครื่อง แต่เป็นแบบ Connectionless คือ ทั้งฝ่ายส่งและฝ่ายรับไม่จำเป็นต้องอาศัยการสร้างช่องทางเชื่อมต่อกัน โดยไม่ต้องมีการแจ้งให้ฝ่ายรับข้อมูลเตรียมรับข้อมูลเหมือนโพรโทคอลทีซีพี และไม่มีการส่งสัญญาณตรวจสอบว่าข้อมูลถึงปลายทางอย่าง

ถูกต้องครบถ้วนสมบูรณ์ ในการส่งข้อมูลแต่ละครั้งจึงไม่มี การส่งข้อมูลใหม่อีกในกรณีที่เกิดความผิดพลาดของการส่งข้อมูล

ส่วนประกอบของยูดีพีเตทาแกรม

|             |    |                  |    |
|-------------|----|------------------|----|
| 0           | 15 | 16               | 31 |
| source port |    | destination port |    |
| length      |    | checksum         |    |
| data        |    |                  |    |

รูปที่ 2-6 ยูดีพีเตทาแกรม

- source port ขนาด 16 บิต: บอกรหัสของเครื่องต้นทาง
- destination port ขนาด 16 บิต: บอกรหัสของเครื่องปลายทาง
- length ขนาด 16 บิต: บอกความยาวของเตทาแกรม (ทั้งเฮดเดอร์และข้อมูล) เป็นจำนวนไบต์
- checksum ขนาด 16 บิต: ผลรวมตรวจสอบ คำนวณจากผลรวมของเฮดเดอร์และข้อมูล

## 2.6 โพรโทคอลไอพี (IP: Internet Protocol)

โพรโทคอลไอพีเป็นโพรโทคอลที่จัดการเกี่ยวกับแอดเดรสของแต่ละและแพ็กเก็ต เพื่อให้ส่งแพ็กเก็ตต่าง ๆ ไปยังเป้าหมายได้อย่างถูกต้อง การทำงานของไอพีเป็นเพียงการส่งข้อมูลไปยังเครื่องเป้าหมายเท่านั้นไม่มีการส่งสัญญาณขอบริการ หรือสัญญาณให้บริการระหว่างกันเหมือนที่ซีพี เรียกว่าการเชื่อมต่อแบบ Connectionless ซึ่งระบบทั้งสองตั้งสมมุติฐานว่าการเชื่อมต่อระหว่างกันไม่มีความผิดพลาดเกิดขึ้น

ส่วนประกอบของไอพีเตทาแกรม

|                        |    |     |          |       |                 |                 |  |
|------------------------|----|-----|----------|-------|-----------------|-----------------|--|
| 0                      | 15 |     |          | 16    | 31              |                 |  |
| version                |    | IHL |          | TOS   |                 | total length    |  |
| identification         |    |     |          | flags |                 | fragment offset |  |
| time to live           |    |     | protocol |       | header checksum |                 |  |
| source IP address      |    |     |          |       |                 |                 |  |
| destination IP address |    |     |          |       |                 |                 |  |
| options                |    |     |          |       | padding         |                 |  |
| data                   |    |     |          |       |                 |                 |  |

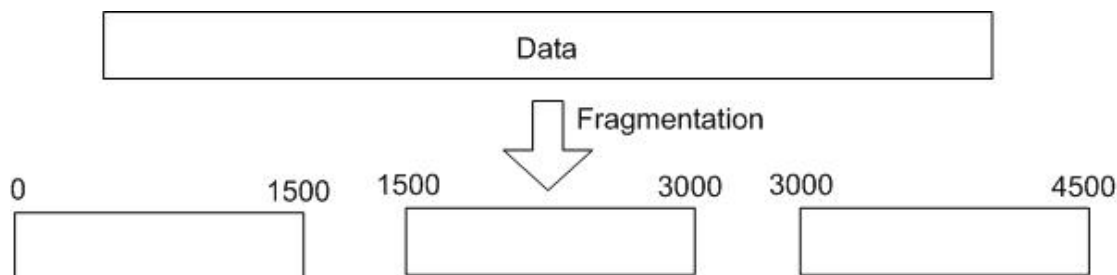
รูปที่ 2-7 ไอพีเตทาแกรม

- version ขนาด 4 บิต: แสดงรุ่นของโปรโตคอล
- Internet Header Length (IHL) ขนาด 4 บิต: บอกความยาวเฉพาะเฮดเดอร์ ของเดทา แกรมโดยนับจาก version จนถึงไบต์สุดท้ายก่อนจะถึงข้อมูล
- Type Of Service (TOS) ขนาด 8 บิต: ฟิลด์นี้ใช้กำหนดรูปแบบการให้บริการตามลักษณะโปรโตคอลแอปพลิเคชัน
- total length มีขนาด 16 บิต: บอกถึงความยาวทั้งหมดของเดทาแกรม
- identification ขนาด 16 บิต
- flags ขนาด 3 บิต
- Time To Live (TTL) ขนาด 8 บิต: ฟิลด์นี้ใช้กำหนดจำนวนเรเตอร์ที่เดทาแกรมจะเดินทางผ่านไปได้อีกหนึ่งหนึ่งคือ กำหนดอายุของเดทาแกรมซึ่งมีค่าสูงสุดตามขนาดฟิลด์คือ  $2^8 - 1 = 255$
- Protocol ขนาด 8 บิต: ฟิลด์บอกชนิดของโปรโตคอลในระดับบนที่เอ็นแคปซูลในเดทาแกรม เพื่อให้สถานีปลายทางสามารถส่งข้อมูลไปยังโปรโตคอลระดับบนได้อย่างถูกต้อง
- header checksum ขนาด 16 บิต: ใช้สำหรับตรวจสอบความผิดพลาดของเฮดเดอร์โดยไม่รวมส่วนของข้อมูล
- source IP address ขนาด 32 บิต: กำหนดไอพีแอดเดรสต้นทาง
- destination IP address ขนาด 32 บิต: กำหนดไอพีแอดเดรสปลายทาง
- option ขนาดไม่คงที่: ใช้สำหรับกำหนดข่าวสารเพิ่มเติมสำหรับเดทาแกรม ค่าที่ใช้ในปัจจุบันเกี่ยวข้องกับการรักษาความปลอดภัย
- padding ขนาด 0 ถึง 3 ไบต์: ใช้สำหรับผนวกเพิ่มเพื่อให้จำนวนไบต์ของ option รวมกับ padding เป็นจำนวนเท่าของ 32 บิต
- data ขนาดไม่คงที่: ข้อมูลของโปรโตคอลระดับบน

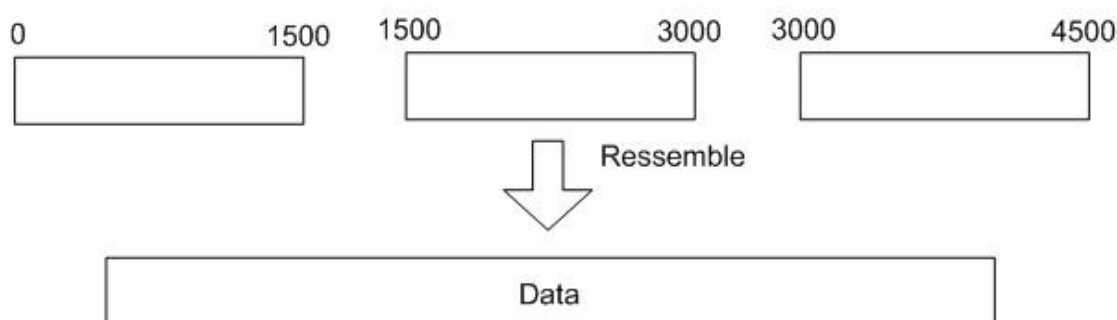
เนื่องจากมาตรฐานในเครือข่ายมีหลากหลาย ขนาดของแพ็กเก็ตเกิดในแต่ละมาตรฐานจึงมีความแตกต่างกันออกไป ทำให้การส่งข้อมูลระหว่างอุปกรณ์ในเครือข่ายนั้นอาจมีการแบ่งข้อมูลออกเป็นแพ็กเก็ตย่อย ๆ ในระหว่างการส่ง เรียกว่าการทำแฟล็กเมนต์ชัน (Fragmentation) เช่นแพ็กเก็ต FDDI มีขนาด 4,500 ไบต์ หากเครื่องปลายทางอยู่ในเครือข่าย Ethernet ซึ่งมีขนาดของแพ็กเก็ตสูงสุด 1,500 ไบต์ ดังนั้นการส่งแพ็กเก็ตไปยังเครื่องปลายทางจึงต้องมีการ แบ่งเป็นแฟ็กเก็ตย่อย ๆ และเมื่อแฟ็กเก็ตย่อยมาถึงเครื่องเป้าหมายก็จะรวมกันเป็นแฟ็กเก็ตเดิมที่มีขนาด 4,500 ไบต์ อีกครั้งหนึ่ง เรียกการรวมกันนี้ว่าการรีแอสเซมเบิล (Reassemble) ซึ่งทำให้ได้ข้อมูลเหมือนกับที่ส่งมาจากเครื่องต้นทาง

เดทาแกรมที่ถูกแฟล็กเมนต์ชันจะมีเฮดเดอร์ประจำตัวเอง โดยมีข้อมูลเพิ่มเติมกำหนดลักษณะของแฟล็กเมนต์ชันประกอบไปด้วย แต่ละชิ้นของแฟล็กเมนต์ชันจะเป็นเดทาแกรมที่สมบูรณ์ในตัวเอง เราเตอร์ระหว่างทางที่รับเดทาแกรมจะไม่รวมเดทาแกรม (Reassembly) แต่จะปล่อยให้ป็นหน้าที่ของเรเตอร์ปลายทาง





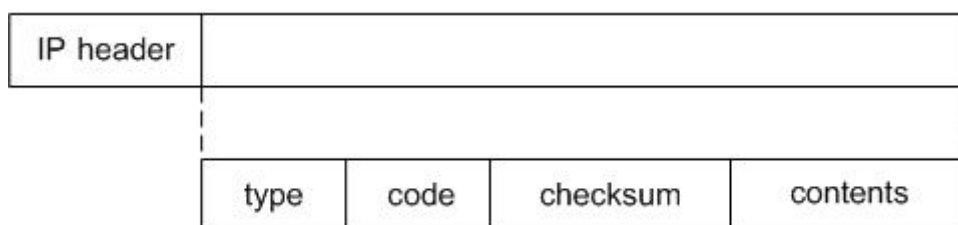
รูปที่ 2-8 แสดงการทำแฟล็กเมนต์ชิ้น



รูปที่ 2-9 แสดงการรีแอสเซมเบิล

## 2.7 โพรโทคอลไอซีเอ็มพี (ICMP: Internet Control Message Protocol)

หน้าที่หลักของโปรโตคอล ไอซีเอ็มพี คือการแจ้งหรือแสดงข้อความจากระบบ เพื่อบอกให้ผู้ใช้ทราบว่าเกิดอะไรขึ้นในการส่งผ่านข้อมูลนั้น ซึ่งปัญหาส่วนมากที่พบคือส่งไปไม่ได้ หรือปลายทางรับข้อมูลไม่ได้ เป็นต้น นอกจากนี้โปรโตคอล ไอซีเอ็มพียังถูกเรียกใช้งานจากเครื่องเซิร์ฟเวอร์ และ เราเตอร์อีกด้วยเพื่อแลกเปลี่ยนข้อมูลที่ใช้ควบคุม ส่วนรูปแบบการทำงานของโปรโตคอลไอซีเอ็มพีนั้น จะทำงานควบคู่กับโปรโตคอลไอพี (IP) ในระดับเดียวกัน และข้อความต่าง ๆ ที่แจ้งให้ทราบจะถูกรวมอยู่ภายในข้อมูลของไอพีเดทาแกรม อีกทีหนึ่ง



รูปที่ 2-10 ข่าวดสารของไอซีเอ็มพีบรรจุในไอพี

ข้อความที่โปรโตคอลไอซีเอ็มพีส่งนั้นแบบออกได้เป็น 2 แบบคือ ICMP error message คือ ข้อความแจ้งข้อผิดพลาด และ ICMP query คือ ข้อความเรียกขอข้อมูลเพิ่มเติม ตัวอย่างกลไกการทำงานของโปรโตคอล

ไอซีเอ็มพีเช่น เมื่อมีการส่งผ่านข้อมูลจากผู้ใช้ไปยังปลายทางที่ไม่ถูกต้อง หรือขณะนั้นเครือข่ายทางเกิดปัญหา จะไม่สามารถรับข้อมูลได้ ที่เราเตอร์จะส่งข้อความ แจ้งเป็น ไอซีเอ็มพี ที่ชื่อ destination unreachable ให้กับผู้ส่งข้อมูล นอกจากนี้ตัวข้อมูลที่แจ้งข้อความก็จะมีส่วนของข้อมูลไอพีเดตาแกรมที่เกิดปัญหาด้วย ดังนั้นเมื่อผู้ส่งข้อมูลได้รับข้อความแจ้งแล้วก็จะได้ทราบว่าจุดที่เกิดปัญหาอยู่ที่ใด

ดังนั้นโปรโตคอล ไอซีเอ็มพี จึงกลายเป็นเครื่องมืออย่างหนึ่งในการช่วยทดสอบเครือข่าย เช่น คำสั่ง ping ที่เรามักใช้ทดสอบว่าเครือข่ายที่ให้บริการหรืออุปกรณ์ที่ต่ออยู่ในเครือข่าย อินเทอร์เน็ต นั้นยังทำงานเป็นปกติหรือไม่ แล้วคำสั่ง ping มีการเรียกใช้งานโปรโตคอล ไอซีเอ็มพี แจ้งเป็นข่าวสารให้ทราบอีกต่อไป

โครงสร้างของไอซีเอ็มพี

|          |      |          |    |
|----------|------|----------|----|
| 0        | 7 8  | 15 16    | 31 |
| type     | code | checksum |    |
| contents |      |          |    |

รูปที่ 2-11 โครงสร้างของไอซีเอ็มพี

- type ขนาด 8 บิต: กำหนดทั้งค่าความผิดพลาดและการรายงานสถานะ การใช้งานในปัจจุบันมีทั้งหมด 15 ประเภท
- code ขนาด 8 บิต: รหัสความผิดพลาด
- checksum ขนาด 16 บิต: ค่าผลรวมตรวจสอบแบบ 1's complement สำหรับตรวจสอบความผิดพลาด โดยคำนวณผลรวมของ type, code และ contents
- contents ขนาดไม่คงที่ : ฟิลด์นี้ใช้บรรจุข้อมูลข่าวสารเพิ่มเติมเพื่อแจ้งกลับซึ่งจะขึ้นอยู่กับค่า type และ code